DevOps Question Bank
Docker & Containers

39 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based
Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
Docker & Containers	17	13	9	39

Questions in this topic

Basic **5**

Q1. What is a Docker image and what is a Docker container? . . . 5

Q2. How do containerisation technologies like Docker and Kubernetes simplify application deployment and management? . . . 5

Q3. What is a Dockerfile? Explain its purpose and structure, and how you write one. . . . 6

Q4. What is WORKDIR in a Dockerfile? What does it do? . . . 7

Q5. How do you define a startup command in Docker? What is the difference between CMD and ENTRYPOINT? . . . 8

Q6. In a Dockerfile, what is the practical difference between COPY and ADD? When would you prefer one over the other? . . . 9

Q7. How does Docker networking work? . . . 9

Q8. What is a multi-stage Docker build, why is it used, and what is its advantage over a single-stage build? . . . 10

Q9. What are the stages or steps of a Docker image build? Explain build context, layers and cache. . . . 11

Q10. What is the docker command to build an image, and which flags are commonly used? . . . 12

Q11. How do you pass environment variables during docker build? What is the difference between ARG and ENV, and how do you handle secrets at build time? . . . 13

Q12. Which container registry do you use, and how do you choose one? . . . 14

Q13. What are Docker labels and why would you use them? . . . 14

Q14. What are Docker restart policies (no, on-failure, always, unless-stopped)? . . . 15

Q15. What is Docker Swarm and how is it different from Kubernetes? . . . 16

Q16. What are the common Docker commands you use daily? . . . 16

Q17. How do volumes work in Docker? When do you use bind mounts vs named volumes (and tmpfs)? 18

Advanced **19**

Q18. Write a sample multi-stage Dockerfile you would actually use in a real project (Node.js app). . . . 19

Q19. Create a Dockerfile for running a database container (MySQL or PostgreSQL). . . . 20

Q20. Write a Dockerfile for a monitoring tool like Nagios. . . . 21

Q21. Besides multi-stage builds and slim base images, how else can you reduce Docker image size? . 22

Q22. How do you manage and push Docker images when you have multiple services? . . . 23

Q23. Do you have an archival or cleanup process for Docker images in AWS ECR? . . . 24

Q24. How do you secure Docker containers and images? . . . 25

Q25. How do you pass database credentials (or other secrets) to a containerised application securely? . 26

Q26. How do you scan Docker images for vulnerabilities, both during the build and at the registry level? 27

Q27. How do you monitor Docker containers in production? . . . 28

Q28. How do you handle logging for containers running in production? . . . 28

Q29. How do you do a zero-downtime update of a service using plain Docker (without Kubernetes)? . . 30

Q30. How do you move containers from one server to another? 30

Scenario-Based **31**

Q31. A Docker container keeps restarting (or suddenly stops / crashes) in production. How do you find the root cause? 31

Q32. Your Docker image size has grown from 300MB to 3GB. How would you investigate and optimize it? 32

Q33. Your Docker image build time has shot up to 25 minutes. How do you optimize it? 33

Q34. You are building a Docker image but requirements.txt (or package.json) keeps installing an old dependency version. How do you fix it? 34

Q35. Docker containers are consuming high CPU on the host. How do you identify the root cause? . . . 35

Q36. Docker shows "no space left on device". How do you resolve it? 35

Q37. The Docker daemon becomes unavailable on a production server. How would you recover? 36

Q38. A containerised app runs perfectly on a developer machine but crashes in production. What is your debug checklist? 37

Q39. A container is running but not reachable on its port. How do you debug it? 38

Docker & Containers

Basic

Q1. What is a Docker image and what is a Docker container?

> **In simple words:** An image is like a recipe card or a frozen ready-meal packet. A container is the actual meal you heat up and eat. One packet design, many meals.

Detailed answer

- A **Docker image** is a read-only, layered package that holds everything an app needs: OS base files, runtime, libraries, your code and the default start command. It is built from a Dockerfile and stored in a registry (Docker Hub, ECR).
- A **Docker container** is a running (or stopped) instance of that image. Docker adds a thin writable layer on top of the image, gives it its own process space, network and filesystem view.
- You can start 10 containers from 1 image. Deleting a container does not delete the image.
- Anything written inside the container's writable layer is lost when the container is removed, unless you use a volume.

Example:

```
docker pull nginx:1.27            # download the image
docker images                     # list images
docker run -d --name web1 -p 8080:80 nginx:1.27   # container 1
docker run -d --name web2 -p 8081:80 nginx:1.27   # container 2, same image
docker ps                         # list running containers
```

> ***What to say to the interviewer:*** *An image is the immutable, versioned blueprint built from a Dockerfile; a container is a running instance of that image with its own writable layer, network and processes. One image can run as many containers, and removing a container never touches the image.*

Q2. How do containerisation technologies like Docker and Kubernetes simplify application deployment and management?

Also asked as: What is the difference between Docker and Kubernetes, and how do they complement each other in production?

> **In simple words:** Shipping containers made global trade easy because every crane and ship handles the same box shape. Docker puts every app in the same "box", and Kubernetes is the port that stacks, moves and replaces those boxes automatically.

Detailed answer

Docker solves "it works on my machine":

- App + dependencies are packed into one image, so dev, test and prod run exactly the same thing.
- Starts in seconds (no full OS boot like a VM), uses fewer resources.
- Versioned images (`app:1.4.2`) make rollback as simple as running the old tag.

Kubernetes solves "running many containers on many servers":

- Scheduling: decides which node each container runs on.
- Self-healing: restarts crashed containers, replaces pods on failed nodes.
- Scaling: HPA adds or removes pods based on load.
- Rolling updates and rollbacks with zero downtime.
- Service discovery and load balancing via Services and Ingress.

Docker vs Kubernetes - not competitors, they complement each other:

Point	Docker	Kubernetes
Job	Build images and run containers	Orchestrate containers across many nodes
Scope	One host	A cluster of many nodes
Failure handling	Restart policy on that host only	Reschedules pods to healthy nodes
Scaling	Manual (`docker run` more copies)	HPA / Cluster Autoscaler, declarative replicas
Networking	Bridge networks, `-p` port mapping	Services, DNS, Ingress, NetworkPolicy
Updates	Stop old, start new yourself	Rolling updates and rollback built in

Runtime detail interviewers like: Kubernetes no longer talks to the Docker daemon - the dockershim was removed in Kubernetes 1.24, so nodes use a CRI runtime such as **containerd** or **CRI-O**. Docker-built images are standard **OCI images**, so they still run unchanged. In practice: Docker (or buildx) builds and pushes the image in CI, Kubernetes runs it in production.

Example:

```
# Docker: package once
docker build -t myorg/orders:1.4.2 .
docker push myorg/orders:1.4.2

# Kubernetes: run 3 copies, update, roll back
kubectl create deployment orders --image=myorg/orders:1.4.2 --replicas=3
kubectl set image deployment/orders orders=myorg/orders:1.4.3
kubectl rollout undo deployment/orders
```

> ***What to say to the interviewer:*** *Docker gives a consistent, portable, versioned package so the same artifact runs everywhere. Kubernetes then orchestrates those containers at scale - scheduling, self-healing, autoscaling, rolling updates and service discovery - so we manage desired state instead of individual servers. Since Kubernetes 1.24 nodes run containers through containerd or CRI-O rather than the Docker daemon, but Docker-built OCI images run unchanged, so Docker builds in CI and Kubernetes runs in production.*

Q3. What is a Dockerfile? Explain its purpose and structure, and how you write one.

Also asked as: What is the structure of your Dockerfile? How do you create a Dockerfile?

In simple words: A Dockerfile is a step-by-step instruction sheet: "start with this base, copy these files, install these things, and when switched on, run this." Docker follows it top to bottom to cook the

image.

Detailed answer

A Dockerfile is a plain text file of instructions. Each instruction creates a layer, and layers are cached, so order matters: put things that change rarely at the top and your source code near the bottom.

Common instructions, in typical order:

- `FROM` - base image (always first).
- `ARG` / `ENV` - build-time and runtime variables.
- `WORKDIR` - working folder inside the image.
- `COPY` - copy dependency files first, then source code.
- `RUN` - run commands at build time (install packages).
- `EXPOSE` - documents the port the app listens on.
- `USER` - switch to a non-root user.
- `HEALTHCHECK` - optional health command.
- `ENTRYPOINT` / `CMD` - what runs when the container starts.

Example:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

```
docker build -t myapp:1.0 .
docker run -d -p 3000:3000 myapp:1.0
```

What to say to the interviewer: *A Dockerfile is the build recipe for an image - base image, working directory, dependency install, code copy, exposed port, user and start command. I order it so rarely changing layers like dependency installs come before the source copy, which keeps builds fast through layer caching.*

Q4. What is WORKDIR in a Dockerfile? What does it do?

In simple words: It is like doing "cd into this folder" before you start working. Every later step happens inside that folder.

Detailed answer

- `WORKDIR /app` sets the current directory for all following `RUN`, `CMD`, `ENTRYPOINT`, `COPY` and `ADD` instructions.
- If the folder does not exist, Docker creates it.
- It is also the default directory when the container starts (and when you `docker exec` into it).

- Prefer `WORKDIR` over `RUN cd /app && ...`, because `cd` in one `RUN` does not carry over to the next `RUN` - each RUN is a fresh shell.
- You can use it multiple times; relative paths build on the previous one.

Example:

```
FROM alpine:3.20
WORKDIR /app          # creates /app and moves into it
COPY . .              # copies into /app
RUN pwd               # prints /app
WORKDIR logs          # now /app/logs
CMD ["sh", "-c", "pwd"]   # container prints /app/logs
```

> ***What to say to the interviewer:*** *WORKDIR sets the working directory for every later instruction and for the running container, creating it if needed. I use it instead of RUN cd because each RUN starts a new shell, so a cd would not persist.*

Q5. How do you define a startup command in Docker? What is the difference between CMD and ENTRYPOINT?

> **In simple words:** ENTRYPOINT is the fixed job of the machine (a coffee machine always makes coffee). CMD is the default setting (medium size) that the user can easily change when pressing the button.

Detailed answer

Both define what runs when the container starts.

Point	CMD	ENTRYPOINT
Purpose	Default command or default arguments	Main executable that always runs
Override at `docker run`	Replaced by any args after image name	Only with `--entrypoint` flag
Used together	Provides default args to ENTRYPOINT	Receives CMD as its arguments
Typical use	Simple apps, easy to override	Wrapper scripts, CLI-style images

- Always use the **exec form** (JSON array) `["node","server.js"]` so your app runs as PID 1 and receives stop signals (SIGTERM). The shell form `CMD node server.js` runs under `/bin/sh -c` and may not get signals, causing slow, ungraceful shutdowns.
- Only the last CMD / ENTRYPOINT in a Dockerfile counts.

Example:

```
FROM alpine:3.20
ENTRYPOINT ["ping"]
CMD ["-c", "3", "google.com"]
```

```
docker run myping                # runs: ping -c 3 google.com
docker run myping -c 1 8.8.8.8   # runs: ping -c 1 8.8.8.8  (CMD replaced)
docker run --entrypoint sh -it myping   # ENTRYPOINT overridden
```

A real-world pair from the official nginx style - the binary is fixed, the flags are the default:

```
FROM nginx:1.27-alpine
ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]
# docker run mynginx -t        -> runs "nginx -t" (config test) instead of the server
```

> ***What to say to the interviewer:*** *ENTRYPOINT defines the fixed executable and CMD supplies default arguments that users can override at docker run. I combine them - ENTRYPOINT for the binary or an entrypoint script, CMD for default flags - and always use the exec form so the process gets signals as PID 1.*

Q6. In a Dockerfile, what is the practical difference between COPY and ADD? When would you prefer one over the other?

> **In simple words:** COPY is a plain photocopy. ADD is a photocopy machine with extra tricks - it can unzip boxes and fetch things from the internet. Extra tricks mean surprises, so use the plain one unless you need the trick.

Detailed answer

Point	COPY	ADD
Copy local files/folders	Yes	Yes
Auto-extract local tar (.tar, .tar.gz)	No	Yes
Download from URL	No	Yes (not auto-extracted)
Predictability	High	Lower (magic behaviour)
Best practice	Default choice	Only for local tar extraction

- Prefer **COPY** - clear and predictable, recommended by Docker.
- Use **ADD** only when you want a local tarball unpacked into the image.
- Instead of `ADD <url>`, use `RUN curl ... && verify checksum && extract && cleanup` in one layer, so the downloaded archive does not bloat the image and you can verify it.

Example:

```
COPY package.json ./                # simple copy
COPY --chown=node:node src/ ./src/   # copy with ownership
ADD rootfs.tar.gz /                 # auto-extracts into /
RUN curl -fsSL -o /tmp/tool.tgz https://example.com/tool.tgz \
 && tar -xzf /tmp/tool.tgz -C /usr/local/bin \
 && rm /tmp/tool.tgz
```

> ***What to say to the interviewer:*** *COPY just copies local files; ADD also extracts local tar archives and can fetch URLs. I use COPY by default and ADD only for local tar extraction; for remote files I use curl in a RUN step so I can verify and clean up in the same layer.*

Q7. How does Docker networking work?

> **In simple words:** Docker builds a small private office network on your server. Containers on the same office network can call each other by name; to let outsiders in, you open a door (publish a port) on the building.

Detailed answer

Docker creates virtual networks using Linux network namespaces, virtual ethernet pairs (veth) and iptables rules.

Network drivers:

- **bridge** (default) - private network on the host (`docker0`, 172.17.0.0/16). Containers get private IPs; outbound traffic is NATed through the host.
- **user-defined bridge** - same as bridge but with built-in DNS, so containers reach each other by container name. Recommended for multi-container apps.
- **host** - container shares the host network stack directly; no isolation, no port mapping, fastest.
- **none** - no networking.
- **overlay** - spans multiple hosts (Docker Swarm).
- **macvlan** - container gets its own MAC/IP on the physical LAN.

Port publishing `-p 8080:80` adds a NAT rule: host port 8080 -> container port 80.

Example:

```
docker network create appnet
docker run -d --name db  --network appnet -e POSTGRES_PASSWORD=secret postgres:16
docker run -d --name api --network appnet -p 8080:3000 myorg/api:1.0
# inside api, the DB is reachable as host "db" on port 5432
docker exec api getent hosts db
docker network inspect appnet
```

> ***What to say to the interviewer:*** *By default containers join a bridge network with private IPs and NAT to the outside; publishing a port with -p maps a host port to the container. For multi-container apps I use a user-defined bridge so containers find each other by name through Docker's embedded DNS, and there are host, none, overlay and macvlan drivers for other cases.*

Q8. What is a multi-stage Docker build, why is it used, and what is its advantage over a single-stage build?

> **In simple words:** You build furniture in a workshop full of saws and glue, but you only deliver the finished table to the customer - not the whole workshop. Multi-stage builds ship only the finished product.

Detailed answer

A multi-stage Dockerfile has more than one `FROM`. Early stages have compilers, build tools and dev dependencies; the final stage copies only the built output with `COPY --from=<stage>`.

Advantages over single-stage:

- **Much smaller images** (for example 1.1 GB -> 150 MB), so faster pulls, faster scaling, less registry cost.
- **Better security** - no compilers, package managers, source code or build secrets in the final image, so fewer CVEs and smaller attack surface.
- **One Dockerfile** replaces separate "build" and "runtime" Dockerfiles or shell scripts.
- **Better caching** - stages build in parallel with BuildKit and unused stages are skipped.
- Can target a stage for testing: `docker build --target test .`

Example:

```
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app

FROM gcr.io/distroless/static-debian12
COPY --from=build /app /app
ENTRYPOINT ["/app"]
# final image ~10 MB instead of ~900 MB
```

> ***What to say to the interviewer:*** *A multi-stage build uses several FROM stages - a heavy build stage and a minimal runtime stage - and copies only the compiled artifact forward. That gives much smaller, more secure images with no build tools or source inside, all from a single Dockerfile.*

Q9. What are the stages or steps of a Docker image build? Explain build context, layers and cache.

> **In simple words:** You hand the chef a box of ingredients (build context), the chef follows the recipe line by line (instructions), puts each finished step on its own tray (layer), and remembers trays already made so it does not redo them next time (cache).

Detailed answer

What happens when you run `docker build -t app:1.0 .`:

1. **Build context is sent** - the `.` folder (minus `.dockerignore` entries) is packed and sent to the Docker daemon / BuildKit. A huge context = slow builds.
2. **Dockerfile is parsed** - `FROM` pulls the base image if not present.
3. **Each instruction runs in order** - `RUN`, `COPY`, `ADD` each produce a new read-only **layer** (a filesystem diff). `ENV`, `CMD`, `EXPOSE`, `LABEL` only change metadata.
4. **Cache check per step** - if the instruction text (and for COPY/ADD, the file checksums) is identical to a previous build, Docker reuses the cached layer. The first step that changes invalidates the cache for **every step after it**.
5. **Image is assembled** - layers stacked + config (entrypoint, env, labels) -> image ID, then tagged.
6. **Push (optional)** - only layers the registry does not already have are uploaded.

That is why you copy dependency files and install them before copying source code.

Example:

```
docker build --progress=plain -t app:1.0 .
#  => [internal] load build context         transferring context: 2.1MB
#  => [2/5] WORKDIR /app                     CACHED
#  => [3/5] COPY package*.json ./            CACHED
#  => [4/5] RUN npm ci                       CACHED
#  => [5/5] COPY . .                         0.3s   <- only this step reran
docker history app:1.0
```

> ***What to say to the interviewer:*** *The build sends the context to the daemon, pulls the base image, then executes each instruction in order, with RUN, COPY and ADD each creating a cached layer. Once a step changes, all later steps rebuild, so I order the Dockerfile from least to most frequently*

changing and keep the context small with .dockerignore.

Q10. What is the docker command to build an image, and which flags are commonly used?

In simple words: "docker build" is the "cook now" button. The flags are the settings: what to name the dish, which recipe card to use, which ingredients to pass in, whether to reuse leftovers, and which kitchen (CPU type) it is cooked for. The dot at the end means "use the ingredients in this folder".

Detailed answer

Basic form: `docker build -t name:tag -f Dockerfile .`

- The trailing `.` is the **build context** - the folder whose files are sent to the builder and that `COPY`/`ADD` can see (filtered by `.dockerignore`). It is not the Dockerfile location.

Flag	What it does
`-t name:tag`	Name and tag the image (repeat for multiple tags)
`-f path/Dockerfile`	Use a Dockerfile other than `./Dockerfile`
`--build-arg KEY=val`	Pass a value to an `ARG`
`--no-cache`	Ignore the layer cache, rebuild every step
`--pull`	Always pull the newest version of the base image
`--target build`	Stop at a named stage of a multi-stage build
`--platform linux/amd64,linux/arm64`	Build for other CPU architectures (multi-arch with `buildx`)
`--secret id=npmrc,src=~/.npmrc`	Mount a build-time secret without storing it in a layer
`--label key=value`	Add metadata (git SHA, team)
`--progress=plain`	Full step-by-step output, useful for debugging slow builds
`--cache-from` / `--cache-to`	Use or export a remote build cache (CI speed-up)
`--push` (buildx)	Push straight to the registry after building

Example:

```
# Everyday build
docker build -t myorg/api:1.4.2 .

# Different Dockerfile, build args, fresh base, label
docker build -f docker/Dockerfile.prod \
  --build-arg APP_VERSION=1.4.2 --pull \
  --label org.opencontainers.image.revision=$(git rev-parse HEAD) \
  -t myorg/api:1.4.2 .

# Build only the test stage
docker build --target test -t myorg/api:test .

# CI: multi-arch, secret, remote cache, push
docker buildx build --platform linux/amd64,linux/arm64 \
  --secret id=npmrc,src=$HOME/.npmrc \
  --cache-from type=registry,ref=$REG/api:buildcache \
  --cache-to type=registry,ref=$REG/api:buildcache,mode=max \
  --progress=plain -t $REG/api:1.4.2 --push .
```

What to say to the interviewer: *The command is docker build -t name:tag -f Dockerfile followed by the build context, usually a dot for the current folder. Common flags are --build-arg, --no-cache and --pull for clean rebuilds, --target for a multi-stage stage, --secret for build-time secrets, --label for metadata, and with buildx --platform for multi-arch plus --cache-from and --cache-to for fast CI builds.*

Q11. How do you pass environment variables during docker build? What is the difference between ARG and ENV, and how do you handle secrets at build time?

In simple words: ARG is a sticky note you use only while cooking and then throw away. ENV is a label printed on the final box that everyone who opens it can read. Secrets should be neither - they are handed over through a private window and never stored.

Detailed answer

Point	ARG	ENV
Available during build	Yes	Yes
Available in running container	No	Yes
Set from CLI	`--build-arg NAME=val`	`-e NAME=val` at `docker run`
Stored in image	Value visible in `docker history`	Stored in image config
Good for	Versions, build flags	Runtime config (`NODE_ENV`, `PORT`)

- An `ARG` declared before `FROM` is only usable in `FROM`; redeclare it after `FROM` to use it in steps.
- To make a build arg available at runtime: `ENV APP_VERSION=$APP_VERSION`.
- **Never pass secrets with ARG or ENV** - they end up in history or config. Use BuildKit secrets: the secret is mounted only for that one RUN step and never written into a layer.

Example:

```
# syntax=docker/dockerfile:1
ARG NODE_VERSION=20
FROM node:${NODE_VERSION}-alpine
ARG APP_VERSION=dev
ENV APP_VERSION=${APP_VERSION} NODE_ENV=production
WORKDIR /app
COPY package*.json ./
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc npm ci --omit=dev
COPY . .
CMD ["node", "server.js"]
```

```
docker build \
  --build-arg NODE_VERSION=20 --build-arg APP_VERSION=1.4.2 \
  --secret id=npmrc,src=$HOME/.npmrc \
  -t app:1.4.2 .
docker run -e PORT=8080 app:1.4.2
```

What to say to the interviewer: *ARG is a build-time-only variable passed with --build-arg, ENV is baked into the image and available at runtime, and I can copy an ARG into an ENV if the app needs it. Secrets never go through either - I use BuildKit --secret mounts so they exist only during that RUN step and never land in a layer.*

Q12. Which container registry do you use, and how do you choose one?

In simple words: A registry is the warehouse where finished image boxes are stored and picked up from. You pick the warehouse closest to your trucks, with good locks and a clean-out policy.

Detailed answer

Common options:

Registry	Best when	Notes
AWS ECR	Workloads on AWS (EKS, ECS, EC2)	IAM auth, private, scan on push, lifecycle policies, cross-region replication
Docker Hub	Public/open-source images	Pull rate limits for anonymous/free users
GHCR (GitHub)	Code and CI on GitHub	Auth with GITHUB_TOKEN, per-repo permissions
GCP Artifact Registry / Azure ACR	GCP / Azure workloads	Same idea as ECR in their cloud
Harbor / Nexus / JFrog	Self-hosted, on-prem, multi-cloud	Built-in scanning, replication, RBAC

Selection criteria: close to the runtime (fast, no egress cost), IAM/SSO integration, private by default, vulnerability scanning, immutable tags, retention policies, replication for DR.

Typical answer for an AWS shop: **ECR** for private app images, with a pull-through cache for Docker Hub base images to avoid rate limits.

Example:

```
aws ecr create-repository --repository-name orders \
  --image-tag-mutability IMMUTABLE \
  --image-scanning-configuration scanOnPush=true
aws ecr get-login-password | docker login --username AWS --password-stdin \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com
docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders:1.4.2
```

What to say to the interviewer: *We run on AWS, so we use private ECR repositories - one per service - with IAM-based auth, immutable tags, scan on push and lifecycle policies. Docker Hub is only for public base images, ideally through an ECR pull-through cache to avoid rate limits.*

Q13. What are Docker labels and why would you use them?

In simple words: Labels are sticky tags on a box - "made on this date, by this team, from this code version". They don't change what is inside, but make boxes easy to find, sort and audit.

Detailed answer

- A label is a key-value metadata pair on an image, container, volume or network.
- Set in the Dockerfile with `LABEL`, or at runtime with `docker run --label`.
- **Traceability** - git commit, build number, version, source repo (standard OCI keys like `org.opencontainers.image.revision`).
- **Ownership** - team, contact, cost centre.
- **Filtering and cleanup** - `docker ps --filter label=...`, prune only certain images.

- **Tool config** - Traefik routing rules, Prometheus/log shipper discovery, Watchtower.
- Labels do not affect runtime behaviour and are not for secrets.

Example:

```
ARG GIT_SHA
LABEL org.opencontainers.image.source="https://github.com/myorg/orders" \
      org.opencontainers.image.revision="${GIT_SHA}" \
      org.opencontainers.image.version="1.4.2" \
      team="payments"
```

```
docker build --build-arg GIT_SHA=$(git rev-parse HEAD) -t orders:1.4.2 .
docker inspect orders:1.4.2 --format '{{json .Config.Labels}}'
docker ps --filter "label=team=payments"
docker image prune -a --filter "label=env=dev"
```

> ***What to say to the interviewer:*** *Labels are key-value metadata on images and containers. I use them to record the git SHA, version and owning team for traceability, to filter and clean up resources, and for tools like Traefik or monitoring agents that discover containers by label.*

Q14. What are Docker restart policies (no, on-failure, always, unless-stopped)?

> **In simple words:** It's the instruction to the building's security guard for when a machine switches off: leave it off, restart it only if it broke, always restart it, or restart it unless a human deliberately turned it off.

Detailed answer

Policy	Restarts when	Survives daemon/host reboot	Typical use
no (default)	Never	No	One-off tasks, debugging
`on-failure[:N]`	Exit code is non-zero (optional max N retries)	Yes, if it had failed	Batch jobs, workers
`always`	Any exit; also after daemon restart even if you stopped it manually	Yes	Critical services
`unless-stopped`	Any exit, except when you stopped it manually	Yes, unless manually stopped	Most long-running services

- Docker adds an increasing delay between restarts (backoff) to avoid tight crash loops.
- A restart policy hides crashes - always monitor the `RestartCount` and alert on it.
- In Kubernetes the pod `restartPolicy` and controllers take over this job.

Example:

```
docker run -d --name api --restart unless-stopped myorg/api:1.4.2
docker run -d --name job --restart on-failure:5 myorg/worker:2.0
docker update --restart always api            # change on a running container
docker inspect api --format '{{.HostConfig.RestartPolicy.Name}} restarts={{.RestartCount}}'
```

> ***What to say to the interviewer:*** *Docker has four restart policies - no, on-failure with an optional retry count, always, and unless-stopped. I use unless-stopped for long-running services and on-failure for jobs, and I alert on restart count so a restart policy never silently hides a crash loop.*

Q15. What is Docker Swarm and how is it different from Kubernetes?

In simple words: Both are managers for a fleet of containers. Swarm is a small, simple family-run bus company - easy to start, fewer rules. Kubernetes is a big city transport authority - more setup, but it handles huge scale, many routes and every kind of emergency.

Detailed answer

- **Docker Swarm** is Docker's built-in orchestrator. `docker swarm init` turns Docker hosts into a cluster of manager and worker nodes; you deploy **services** (with replicas) and **stacks** (Compose files) and Swarm keeps them running, load-balances them and does rolling updates.
- **Kubernetes** is a separate, much richer orchestration platform (API server, etcd, scheduler, controllers) and is the industry standard, with managed versions like EKS, GKE and AKS.

Point	Docker Swarm	Kubernetes
Setup	One command, built into Docker	More components; usually managed (EKS/GKE/AKS)
Learning curve	Low - uses Compose files	Steep - many object types
Unit of deployment	Service / task	Pod, Deployment, StatefulSet, etc.
Autoscaling	No built-in autoscaling	HPA, VPA, Cluster Autoscaler, KEDA
Networking	Overlay network + routing mesh	CNI plugins, Services, Ingress, NetworkPolicy
Storage	Basic volumes	PV/PVC, StorageClasses, CSI drivers
Config and secrets	Docker configs and secrets	ConfigMaps, Secrets, external secret operators
Ecosystem	Small, limited active development	Huge: Helm, Argo CD, Prometheus, service mesh
Best for	Small teams, simple apps, few hosts	Production at scale, microservices, multi-team

Example:

```
docker swarm init --advertise-addr 10.0.1.10
docker swarm join-token worker            # command to run on worker nodes
docker service create --name web --replicas 3 -p 80:80 nginx:1.27
docker service scale web=5
docker service update --image nginx:1.27.1 --update-order start-first web
docker stack deploy -c docker-compose.yml shop
```

What to say to the interviewer: *Docker Swarm is Docker's native orchestrator - very easy to set up and uses Compose files, but it has no built-in autoscaling and a small ecosystem. Kubernetes needs more setup but gives autoscaling, rich networking and storage, RBAC and a huge ecosystem, which is why production microservice platforms standardise on it.*

Q16. What are the common Docker commands you use daily?

In simple words: Like a mechanic's most-used tools on the belt - a small set of commands covers 90 percent of daily work: build, run, look inside, read logs, clean up.

Detailed answer

Group	Command	What it does
Images	`docker build -t app:1.0 .`	Build an image from a Dockerfile
Images	`docker images`	List local images
Images	`docker pull` / `docker push`	Download / upload an image
Images	`docker tag app:1.0 $REG/app:1.0`	Add a registry tag
Images	`docker history app:1.0`	Show layers and their sizes
Containers	`docker run -d --name web -p 8080:80 nginx`	Start a container
Containers	`docker ps` / `docker ps -a`	Running / all containers
Containers	`docker stop` / `start` / `restart web`	Control lifecycle
Containers	`docker rm -f web`	Remove a container
Debug	`docker logs -f --tail 100 web`	Follow logs
Debug	`docker exec -it web sh`	Shell inside a running container
Debug	`docker inspect web`	Full JSON: exit code, IP, mounts, env
Debug	`docker stats` / `docker top web`	Live CPU and memory / processes
Debug	`docker events`	Real-time daemon events (die, oom)
Cleanup	`docker system df`	Disk usage by Docker
Cleanup	`docker system prune`	Remove stopped containers, dangling images, unused networks
Cleanup	`docker image prune -a` / `docker builder prune`	Unused images / build cache
Networks	`docker network ls` / `docker network create appnet`	List / create networks
Networks	`docker network inspect appnet`	Which containers and IPs are on it
Volumes	`docker volume ls` / `docker volume create data`	List / create volumes
Volumes	`docker volume inspect data`	Mountpoint on the host
Compose	`docker compose up -d` / `docker compose down`	Start / stop a multi-container stack
Compose	`docker compose ps` / `docker compose logs -f api`	Status / logs for a service
Compose	`docker compose build` / `docker compose pull`	Rebuild / pull service images

Example:

```
docker build -t myorg/api:1.4.2 .
docker run -d --name api --network appnet -p 8080:3000 myorg/api:1.4.2
docker logs -f --tail 50 api
docker exec -it api sh
docker inspect api --format '{{.State.Status}} {{.State.ExitCode}}'
docker system df && docker image prune -a -f --filter "until=168h"
```

What to say to the interviewer: *Day to day I use docker build, tag and push for images; run, ps, stop and rm for containers; logs, exec, inspect, stats and events for debugging; system df and prune for cleanup; plus docker compose up, logs and down for multi-container stacks.*

Q17. How do volumes work in Docker? When do you use bind mounts vs named volumes (and tmpfs)?

In simple words: A container is like a hotel room - when you check out, everything left inside is thrown away. A volume is your own locker that survives after checkout. A bind mount is a window into a specific folder of the host's house. tmpfs is a whiteboard in the room - fast, but wiped when you leave.

Detailed answer

Data written into the container's writable layer is lost when the container is removed and is slow for heavy writes. Mounts keep data outside the container:

Type	Where data lives	Managed by	Best for	Watch out
Named volume	`/var/lib/docker/volumes/<name>/_data`	Docker	Databases, app data in prod	Must back up separately
Bind mount	Any host path you choose	You	Dev live code reload, config files, host logs	Depends on host folder layout and permissions
tmpfs	Host memory only	Docker	Temp files, secrets scratch, read-only containers	Lost on stop, uses RAM

- Named volumes are portable, can use volume drivers (NFS, EBS plugins) and are pre-filled with the image's files at that path on first use.
- A bind mount hides whatever the image had at that path.
- Add `:ro` / `readonly` for config you don't want the app to change.

`-v` vs `--mount`: `-v` is short and silently creates a missing host folder; `--mount` is explicit key=value and errors if the source is missing - clearer for production scripts.

Permissions / UID gotcha: Files are owned by numeric UIDs, not names. If the container runs as UID 1000 but the host folder is owned by root, the app gets `Permission denied`. Fix: `chown` the host folder to the container UID, run the container with `--user $(id -u):$(id -g)` for bind mounts, or set ownership in the Dockerfile (`COPY --chown`, `RUN chown`) before the volume is first created.

Example:

```
# Named volume
docker volume create pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16
docker run -d --name db2 --mount type=volume,src=pgdata2,dst=/var/lib/postgresql/data postgres:16

# Bind mount (read-only config) and tmpfs
docker run -d --name web \
  --mount type=bind,src=/etc/myapp/nginx.conf,dst=/etc/nginx/nginx.conf,readonly \
  --mount type=tmpfs,dst=/tmp,tmpfs-size=64m nginx:1.27

# Back up and restore a volume through a throwaway container
docker run --rm -v pgdata:/data:ro -v $PWD:/backup alpine tar czf /backup/pgdata.tgz -C /data .
docker run --rm -v pgdata:/data -v $PWD:/backup alpine tar xzf /backup/pgdata.tgz -C /data

# UID gotcha fix for a bind mount
sudo chown -R 1000:1000 /srv/app-data
docker volume inspect pgdata --format '{{.Mountpoint}}'
```

For databases, a file-level tar is only safe with the DB stopped - otherwise use the DB's own dump tool (`pg_dump`) or storage snapshots.

> ***What to say to the interviewer:*** *Named volumes are Docker-managed storage that outlives the container, which I use for databases and app data; bind mounts map a specific host path, which I use for config files and local development; tmpfs is in-memory for temporary or sensitive scratch data. I prefer --mount in production because it is explicit, back volumes up through a throwaway container or DB dumps, and handle the UID mismatch by chowning to the container user.*

Advanced

Q18. Write a sample multi-stage Dockerfile you would actually use in a real project (Node.js app).

> **In simple words:** Stage 1 installs everything and builds the app, stage 2 keeps only production packages, and the last stage is a clean, small box with just what is needed to run.

Detailed answer

Real-project points shown below:

- Pin base image versions, never `latest`.
- Copy `package*.json` first so `npm ci` is cached unless dependencies change.
- Separate build stage (dev dependencies, TypeScript compile) from runtime.
- Runtime runs as non-root `node` user, with `NODE_ENV=production`.
- Exec form CMD so SIGTERM reaches Node for graceful shutdown.
- A `.dockerignore` keeps `node_modules`, `.git` and `.env` out of the build context.

Example:

```
# syntax=docker/dockerfile:1
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci

FROM deps AS build
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:20-alpine AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/dist ./dist
COPY --chown=node:node package.json ./
USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

```
# .dockerignore
node_modules
.git
.env
*.log
coverage
```

> ***What to say to the interviewer:*** *I use a deps stage to cache npm ci, a build stage that compiles and prunes dev dependencies, and a slim alpine runtime stage that copies only dist and production node_modules, runs as the non-root node user and uses exec-form CMD. Combined with a .dockerignore, this typically cuts the image by 60-80 percent.*

Q19. Create a Dockerfile for running a database container (MySQL or PostgreSQL).

> **In simple words:** The official database image is a ready-made kitchen. You just add your own recipe book (init scripts) and settings, and keep the food (data) in a separate fridge (volume) so it survives if the kitchen is rebuilt.

Detailed answer

Key points:

- Build on the official image (`postgres:16`, `mysql:8.4`) - do not install a DB from scratch.
- Scripts placed in `/docker-entrypoint-initdb.d/` run **only on first start** when the data directory is empty.
- Never bake passwords into the image; pass them at runtime (env var, or better `*_FILE` pointing to a secret file).
- Always mount a **volume** for the data directory, otherwise data is lost when the container is removed.
- Add a healthcheck so dependent services wait until the DB is ready.

Example:

```
FROM postgres:16-alpine
ENV POSTGRES_DB=appdb \
    POSTGRES_USER=appuser
COPY init/01-schema.sql /docker-entrypoint-initdb.d/
COPY config/postgresql.conf /etc/postgresql/postgresql.conf
HEALTHCHECK --interval=10s --timeout=5s --retries=5 \
  CMD pg_isready -U appuser -d appdb || exit 1
EXPOSE 5432
CMD ["postgres", "-c", "config_file=/etc/postgresql/postgresql.conf"]
```

```
docker build -t mydb:1.0 .
docker run -d --name db \
  -e POSTGRES_PASSWORD_FILE=/run/secrets/db_pass \
  -v /secure/db_pass:/run/secrets/db_pass:ro \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 mydb:1.0
```

> ***What to say to the interviewer:*** *I extend the official Postgres or MySQL image, add init SQL into docker-entrypoint-initdb.d, a custom config and a pg_isready healthcheck, and pass the password at runtime via a secret file rather than baking it in. Data always goes to a named volume; in production I would still prefer a managed DB like RDS.*

Q20. Write a Dockerfile for a monitoring tool like Nagios.

> **In simple words:** Take a clean Ubuntu box, install the Nagios package and its web server from the official package store, add your own list of servers to watch, open the web page door (port 80), and add a self-check so Docker knows it is alive.

Detailed answer

Design points:

- Build on a pinned OS base (`ubuntu:24.04`) and install `nagios4` from the distro repository - no compiling from source. (For a quick start, the widely used community image `jasonrivers/nagios` is an alternative.)
- Nagios Core has two parts: the **nagios4 daemon** (runs the checks) and the **web UI** (Apache + CGI). The container starts both: Nagios in the background, Apache in the foreground as the main process.
- Host and service definitions go into `/etc/nagios4/conf.d/` - copy them in, or better, mount them so config changes do not need a rebuild.
- Volumes for state and logs (`/var/lib/nagios4`, `/var/log/nagios4`).
- The web login password comes from a runtime secret, never baked into the image.
- Validate the config at startup (`nagios4 -v`) so a bad config fails fast.

Example:

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update \
 && apt-get install -y --no-install-recommends nagios4 monitoring-plugins \
      apache2 apache2-utils curl procps \
 && a2enmod cgi rewrite \
 && rm -rf /var/lib/apt/lists/*
COPY conf.d/ /etc/nagios4/conf.d/
COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh
RUN chmod +x /usr/local/bin/docker-entrypoint.sh
VOLUME ["/var/lib/nagios4", "/var/log/nagios4"]
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD pgrep -x nagios4 >/dev/null && curl -s -o /dev/null http://localhost/nagios4/ || exit 1
ENTRYPOINT ["/usr/local/bin/docker-entrypoint.sh"]
```

```
#!/bin/sh
# docker-entrypoint.sh
set -e
if [ -f /run/secrets/nagios_pass ]; then
  htpasswd -bc /etc/nagios4/htpasswd.users nagiosadmin "$(cat /run/secrets/nagios_pass)"
fi
/usr/sbin/nagios4 -v /etc/nagios4/nagios.cfg      # fail fast on bad config
/usr/sbin/nagios4 -d /etc/nagios4/nagios.cfg      # daemon in background
exec apachectl -D FOREGROUND                      # web UI as the main process
```

```
docker build -t mynagios:1.0 .
docker run -d --name nagios -p 8080:80 \
  -v $PWD/conf.d:/etc/nagios4/conf.d:ro \
  -v nagios-data:/var/lib/nagios4 \
  -v /secure/nagios_pass:/run/secrets/nagios_pass:ro mynagios:1.0
# UI: http://<host>:8080/nagios4/
```

> ***What to say to the interviewer:*** *I base it on a pinned Ubuntu image, install nagios4 with Apache from the distro repo, copy or mount the host and service definitions into conf.d, expose port 80 and add a healthcheck that confirms both the daemon and the web UI. An entrypoint script validates the config, starts Nagios in the background and runs Apache in the foreground, with the admin password injected at runtime.*

Q21. Besides multi-stage builds and slim base images, how else can you reduce Docker image size?

Also asked as: How do you optimize a Docker image for production?

> **In simple words:** Packing a suitcase: don't pack what you won't use, throw away the packaging, and don't leave the rubbish inside the bag.

Detailed answer

1. `.dockerignore` - exclude `.git`, `node_modules`, test data, logs, local builds.
2. **Clean up in the same RUN layer** - files deleted in a later layer still exist in the earlier layer.
3. **Skip extras** - `apt-get install --no-install-recommends`, `pip install --no-cache-dir`, `apk add --no-cache`, `npm ci --omit=dev`.
4. **Combine RUN steps** to reduce layers and leftover caches.
5. **Distroless or scratch** final images for compiled apps.

6. **Remove build secrets and caches** - use BuildKit `--mount=type=cache` so caches never land in the image.
7. **Copy only the needed artifact**, not the whole repo.
8. **Analyse layers** with `docker history` and the `dive` tool to find big layers.
9. **Strip binaries / static linking** (Go `-ldflags="-s -w"`).
10. **Squash / fewer COPY of large assets**; serve big static files from S3/CDN instead of the image.

Example:

```
# BAD - cache stays in an earlier layer
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOOD - one layer, cleaned up
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl ca-certificates \
 && rm -rf /var/lib/apt/lists/*
```

```
docker history --no-trunc myapp:1.0 | head
dive myapp:1.0
```

> ***What to say to the interviewer:*** *Beyond multi-stage and slim bases, I use a strict .dockerignore, install with no-recommends and no-cache flags, clean up in the same RUN layer, use distroless or scratch where possible and copy only the final artifact. Then I verify with docker history or dive to find any fat layer.*

Q22. How do you manage and push Docker images when you have multiple services?

Also asked as: How do you connect Jenkins and push Docker images to AWS ECR (Docker side)?

> **In simple words:** Like a warehouse where every product has its own shelf (repository) and every batch gets a unique label (tag), so you always know exactly which version is shipped where.

Detailed answer

- **One repository per service** in the registry (ECR): `orders`, `payments`, `users`.
- **Immutable, traceable tags**: git commit SHA and/or semantic version (`1.4.2`, `sha-3f9c1ab`). Avoid relying on `latest` in prod. Turn on ECR tag immutability.
- **Build once, promote everywhere** - the same image digest moves dev -> staging -> prod; only config changes.
- **CI loops** over changed services only (monorepo path filters) and builds them in parallel.
- **Authenticate with short-lived tokens** (`aws ecr get-login-password` using an IAM role) - no static keys.
- **Scan on push** (ECR scanning or Trivy in CI) and add lifecycle policies to clean old images.
- Use `docker buildx` for multi-arch (amd64/arm64) when needed.

Example:

```
ACCOUNT=123456789012; REGION=ap-south-1
REG=$ACCOUNT.dkr.ecr.$REGION.amazonaws.com
TAG=$(git rev-parse --short HEAD)

aws ecr get-login-password --region $REGION | docker login --username AWS --password-stdin $REG

for svc in orders payments users; do
  docker build -t $REG/$svc:$TAG -t $REG/$svc:1.4.2 ./services/$svc
  docker push $REG/$svc:$TAG
  docker push $REG/$svc:1.4.2
done
```

> ***What to say to the interviewer:*** *Each service gets its own ECR repository and every build is tagged with the git SHA plus a version, with tag immutability on. CI builds only changed services, logs in with short-lived tokens from an IAM role, scans on push and promotes the same image digest across environments.*

Q23. Do you have an archival or cleanup process for Docker images in AWS ECR?

> **In simple words:** A cupboard that throws out old clothes automatically - keep the last few outfits and the special ones, bin the rest - so it never overflows or costs more rent.

Detailed answer

Yes - with **ECR lifecycle policies**. Rules are evaluated by priority and expire images automatically.

Typical policy:

- Keep the last N (for example 20) release-tagged images (`v*` or `prod-*`).
- Delete untagged images (left behind when a tag moves) after 1-7 days.
- Delete dev/feature-branch images older than 14-30 days.

Also:

- Use **tag immutability** so a tag cannot be silently overwritten.
- For long-term audit needs, keep release images in a separate "archive" repository (or replicate to another region/account) with a longer retention rule.
- Test with the lifecycle policy **preview** before applying.
- Clean up locally on build agents too: `docker image prune -a --filter "until=168h"`.

Example:

```
{
  "rules": [
    { "rulePriority": 1, "description": "Keep last 20 releases",
      "selection": { "tagStatus": "tagged", "tagPrefixList": ["v"],
                     "countType": "imageCountMoreThan", "countNumber": 20 },
      "action": { "type": "expire" } },
    { "rulePriority": 2, "description": "Expire untagged after 7 days",
      "selection": { "tagStatus": "untagged", "countType": "sinceImagePushed",
                     "countUnit": "days", "countNumber": 7 },
      "action": { "type": "expire" } }
  ]
}
```

```
aws ecr put-lifecycle-policy --repository-name orders \
  --lifecycle-policy-text file://policy.json
```

> ***What to say to the interviewer:*** *Yes, we use ECR lifecycle policies - keep the last 20 release tags, expire untagged images after a week and feature-branch images after a few weeks - with tag immutability enabled. Anything that must be kept long-term goes to a separate archive repository with its own retention.*

Q24. How do you secure Docker containers and images?

Also asked as: Container security - non-root, distroless, read-only filesystems. How do you optimize (harden) a Docker image for production? / What type of Docker images should be used in production?

> **In simple words:** Lock the house (non-root, read-only), keep few doors and windows (minimal image), and have an inspector check it for known weak locks before anyone moves in (image scanning).

Detailed answer

Build time:

- Minimal, pinned base images (distroless, alpine, slim), ideally pinned by digest.
- **Scan images** in CI and at the registry (details in the image-scanning question below).
- No secrets in images or layers (use BuildKit `--mount=type=secret`).
- Lint Dockerfiles with `hadolint`; sign images (cosign) and generate an SBOM.

Run time:

- Run as **non-root** (`USER 1000`).
- **Read-only root filesystem**, with tmpfs for scratch folders.
- Drop Linux capabilities (`--cap-drop ALL`), `--security-opt no-new-privileges`.
- Never use `--privileged` or mount `/var/run/docker.sock` into app containers.
- Set CPU/memory limits; keep Docker engine and host patched.

Example:

```
trivy image --severity HIGH,CRITICAL --exit-code 1 --ignore-unfixed myapp:1.0
grype myapp:1.0 --fail-on high

docker run -d --name api \
  --user 1000:1000 --read-only --tmpfs /tmp \
  --cap-drop ALL --security-opt no-new-privileges \
  --memory 512m --cpus 1 myapp:1.0
```

Which images to use in production:

- Start from **official, verified-publisher, or an internal golden base image**, never random images from Docker Hub.
- Keep it **minimal:** distroless, `-slim`, alpine (watch musl compatibility) or Ubuntu chiseled. Fewer packages means fewer CVEs and a smaller attack surface.
- **Pin versions:** `node:20.11-slim@sha256:...`, never `latest`, so builds are reproducible.
- Use **multi-stage builds** so compilers and build tools stay out of the final image.

- Run as **non-root**, with no secrets baked in, scanned (Trivy) and signed (cosign), with an SBOM and provenance labels.
- **Rebuild regularly** (for example weekly) so base-image security patches get in, even when the code didn't change.

> ***What to say to the interviewer:*** *I use minimal pinned images, scan every build with Trivy and fail the pipeline on fixable high or critical CVEs, and keep secrets out of layers. At runtime containers run as non-root with a read-only filesystem, dropped capabilities, no-new-privileges and resource limits - never privileged.*

Q25. How do you pass database credentials (or other secrets) to a containerised application securely?

> **In simple words:** Don't write the house key on the front door (the image). Hand it to the person only when they arrive, from a locked key box (secrets manager).

Detailed answer

What NOT to do:

- Hardcode in code, `ENV` in the Dockerfile, or `ARG` (visible in `docker history`).
- Commit `.env` files to git.

Better options, from okay to best:

1. **Runtime env vars** from a protected source (`--env-file` not in git). Simple, but visible in `docker inspect`.
2. **Secret files mounted read-only** (Docker secrets, Compose secrets, Kubernetes Secrets as volumes) and use `*_FILE` variables.
3. **External secret manager** - AWS Secrets Manager / SSM Parameter Store / Vault. The app (or an init step) fetches the secret at startup using its IAM role, so no long-lived credentials exist anywhere. Supports rotation.
4. In Kubernetes: External Secrets Operator or Secrets Store CSI driver syncing from Secrets Manager.
5. For build-time secrets (private package tokens) use BuildKit `--mount=type=secret` so they never land in a layer.

Example:

```
# Docker Compose secret mounted as a file
services:
  app:
    image: myorg/app:1.0
    environment:
      DB_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
secrets:
  db_password:
    file: ./secrets/db_password.txt   # not committed to git
```

```
# Fetch from AWS Secrets Manager at startup using the instance/task IAM role
export DB_PASSWORD=$(aws secretsmanager get-secret-value \
  --secret-id prod/app/db --query SecretString --output text | jq -r .password)
```

What to say to the interviewer: *I never bake credentials into the image or git. In production the secret lives in AWS Secrets Manager or Vault and is injected at runtime - fetched via the workload's IAM role or mounted as a read-only file - which also enables rotation without rebuilding the image.*

Q26. How do you scan Docker images for vulnerabilities, both during the build and at the registry level?

Also asked as: What methods do you use to check container vulnerabilities? Scanning with Trivy or Grype.

In simple words: Two checkpoints - a security check at the factory gate before the box leaves (CI scan), and a warehouse inspector who keeps re-checking stored boxes as new weaknesses are discovered (registry scan).

Detailed answer

1. During the build (CI gate - shift left):

- Run **Trivy**, **Grype** or **Docker Scout** right after `docker build`, before `docker push`.
- Fail the pipeline on **CRITICAL/HIGH fixable** CVEs (`--ignore-unfixed` avoids blocking on issues with no patch).
- Also scan the Dockerfile/IaC for misconfig (`trivy config`, `hadolint`) and generate an SBOM.
- Keep a reviewed `.trivyignore` with expiry dates for accepted risks.

2. At the registry (continuous):

- **ECR basic scanning** - scan on push (Clair-based CVE database).
- **ECR enhanced scanning** via **Amazon Inspector** - continuous re-scanning when new CVEs are published, OS + language packages, findings to Security Hub/EventBridge -> alerts.
- Docker Hub / Harbor / GHCR offer similar scanning.

3. At deploy/runtime: admission policy (Kyverno/OPA) to block unscanned or unsigned images; rebuild base images regularly to pick up patches.

Example:

```
docker build -t $REG/orders:$TAG .
trivy image --severity CRITICAL,HIGH --ignore-unfixed --exit-code 1 $REG/orders:$TAG
grype $REG/orders:$TAG --fail-on critical
docker scout cves $REG/orders:$TAG --only-severity critical,high
docker push $REG/orders:$TAG

aws ecr put-registry-scanning-configuration --scan-type ENHANCED \
  --rules '[{"scanFrequency":"CONTINUOUS_SCAN","repositoryFilters":[{"filter":"*","filterType":"WILD
    CARD"}]}]'
aws ecr describe-image-scan-findings --repository-name orders --image-id imageTag=$TAG
```

What to say to the interviewer: *I scan twice - in CI with Trivy or Grype right after the build, failing the pipeline on fixable critical and high CVEs before the image is pushed, and in the registry with ECR enhanced scanning through Inspector, which keeps re-scanning stored images as new CVEs appear and raises alerts. Base images are rebuilt regularly so fixes actually land.*

Q27. How do you monitor Docker containers in production?

In simple words: Give every container a fitness tracker (metrics), a diary that is collected centrally (logs), and a heartbeat check (health check), then set alarms when something looks wrong.

Detailed answer

- **Metrics**: **cAdvisor** exposes per-container CPU, memory, network, disk I/O; **node-exporter** for the host; **Prometheus** scrapes both; **Grafana** dashboards. `docker stats` for a quick live view.
- **Logs**: containers log to stdout/stderr; a **log driver** (json-file with rotation, awslogs, fluentd) or an agent like **Fluent Bit / Promtail** ships them to Loki, ELK or CloudWatch.
- **Health**: HEALTHCHECK in the Dockerfile -> status shows `healthy/unhealthy`; external uptime checks on the endpoint.
- **Events**: `docker events` for die/oom/restart.
- **Alerts** (Alertmanager -> Teams/Slack/pager): container down, restart count increasing, OOM kills, memory near limit, CPU throttling, unhealthy status, host disk above 80 percent.

Example:

```
docker run -d --name cadvisor -p 8081:8080 \
  -v /:/rootfs:ro -v /var/run:/var/run:ro -v /sys:/sys:ro \
  -v /var/lib/docker/:/var/lib/docker:ro gcr.io/cadvisor/cadvisor:v0.49.1
```

```
# Prometheus alert: container memory above 90 percent of its limit
- alert: ContainerMemoryHigh
  expr: container_memory_working_set_bytes{name!=""}
        / container_spec_memory_limit_bytes{name!=""} > 0.9
  for: 5m
  labels: { severity: warning }
```

What to say to the interviewer: *I use cAdvisor and node-exporter scraped by Prometheus with Grafana dashboards for metrics, Fluent Bit or Promtail shipping stdout logs to Loki or CloudWatch, and Dockerfile HEALTHCHECKs. Alertmanager notifies on container down, rising restart count, OOM kills, memory near limit and host disk usage.*

Q28. How do you handle logging for containers running in production?

In simple words: Every container just talks out loud (stdout). Docker writes it into a notebook that is torn out and replaced when full (rotation), and a courier (Fluent Bit) carries copies to a central library (Loki/ELK/CloudWatch) where anyone can search them - but nobody should ever say passwords out loud.

Detailed answer

1. Log to stdout/stderr, not files inside the container. Docker captures both streams; files inside the container are lost with it and fill the writable layer.

2. Pick a logging driver:

Driver	Where logs go	Note
`json-file`	Local JSON files (default)	No rotation unless configured
`local`	Local, compressed, rotated by default	Good default for single hosts

Driver	Where logs go	Note
`journald`	systemd journal	`journalctl CONTAINER_NAME=api`
`awslogs`	CloudWatch Logs	Needs IAM role on the host
`fluentd`	Fluentd / Fluent Bit endpoint	Central pipelines
`gelf`	Graylog / Logstash	UDP or TCP

`docker logs` works directly with json-file, local and journald; for remote drivers Docker keeps a local dual-logging cache (Docker 20.10+).

3. Rotate logs - default json-file grows forever and can fill `/var/lib/docker`. Set `max-size` and `max-file` in `/etc/docker/daemon.json` (applies to new containers).

4. Ship centrally - an agent (Fluent Bit, Vector, Promtail) tails the container log files and sends them to Loki, ELK/OpenSearch or CloudWatch, adding labels like container, service, env.

5. Structured JSON logs with timestamp, level, service and a `trace_id` so one request can be followed across services and linked to traces.

6. Never log secrets (passwords, tokens, card numbers); mask them in the app and add redaction filters in the shipper.

7. Blocking vs non-blocking mode - default `mode=blocking`: if the log destination is slow, the app's writes to stdout block and the app slows down. `mode=non-blocking` with `max-buffer-size` keeps the app fast, but logs can be dropped when the buffer fills. Use non-blocking for remote drivers on latency-critical apps.

For metrics and alerting on containers, see the monitoring question above - logs explain why, metrics tell you when.

Example:

```
# /etc/docker/daemon.json  (restart docker; affects newly created containers)
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "50m", "max-file": "5", "mode": "non-blocking",
                "max-buffer-size": "4m" }
}
```

```
docker run -d --name api \
  --log-driver awslogs \
  --log-opt awslogs-region=ap-south-1 \
  --log-opt awslogs-group=/prod/api \
  --log-opt mode=non-blocking --log-opt max-buffer-size=4m \
  myorg/api:1.4.2

docker inspect api --format '{{.HostConfig.LogConfig.Type}}'
```

```
{"ts":"2026-09-24T10:15:02Z","level":"error","service":"orders",
 "trace_id":"4bf92f3577b34da6","msg":"payment timeout","duration_ms":3012}
```

> ***What to say to the interviewer:*** *Apps log structured JSON with a trace_id to stdout, Docker's json-file or local driver has max-size and max-file rotation in daemon.json so disks never fill, and Fluent Bit or Promtail ships the logs to Loki, ELK or CloudWatch. I keep secrets out of logs, and for*

remote drivers on latency-sensitive services I use non-blocking mode with a buffer, accepting that some logs may drop under pressure.

Q29. How do you do a zero-downtime update of a service using plain Docker (without Kubernetes)?

In simple words: Open the new shop counter next door, check it works, then move the "Open" sign to it and only then close the old counter. Customers never find a closed door.

Detailed answer

Plain `docker stop` + `docker run` always causes a gap, so use **blue-green behind a reverse proxy** (Nginx, Traefik, HAProxy):

1. Current version (**blue**) runs behind Nginx on port 8081.
2. Start the new version (**green**) on port 8082 with the new image.
3. Wait until green passes its health check.
4. Switch the Nginx upstream to green and `nginx -s reload` (graceful - existing connections finish).
5. Watch errors/latency for a few minutes.
6. Stop blue (keep the image for instant rollback). Rollback = switch upstream back.

Alternatives: Docker Swarm `docker service update --update-order start-first`, or Traefik with labels which discovers containers automatically. The app must handle SIGTERM gracefully and be stateless (sessions in Redis, not memory).

Example:

```
docker run -d --name app-green --network web -p 8082:3000 myorg/app:1.5.0
until curl -fs http://localhost:8082/health; do sleep 2; done

sed -i 's/127.0.0.1:8081/127.0.0.1:8082/' /etc/nginx/conf.d/app.conf
nginx -t && nginx -s reload

sleep 300 && docker stop app-blue     # after verification
```

What to say to the interviewer: *With plain Docker I use blue-green behind Nginx or Traefik - start the new container alongside the old one, wait for its health check, switch the proxy upstream with a graceful reload, then retire the old one while keeping it for fast rollback. Docker Swarm's start-first update order gives the same result natively.*

Q30. How do you move containers from one server to another?

In simple words: You don't move a running kitchen - you send the recipe (image) to the new place, carry the food store (volume data) separately, and start cooking there.

Detailed answer

Containers are disposable - you move the **image**, the **config** and the **data**, then start a fresh container.

1. **Image** - best: push to a registry and pull on the new host. No registry / air-gapped: `docker save` -> copy the tar -> `docker load`.

2. **Config** - `docker run` flags, env files, Compose file (keep them in git). `docker inspect` on the old host shows ports, env, mounts.
3. **Data** - volumes are NOT part of the image or `docker commit`. Back up the volume (tar through a helper container, or DB dump) and restore on the new host.
4. Start on the new host, verify, switch DNS/load balancer, then remove the old one.

Avoid `docker export/import` (flattens the filesystem and loses image metadata like CMD, ENV) and avoid `docker commit` as a migration method (unreproducible images).

Example:

```
# Image via registry
docker push $REG/app:1.4.2          # old host
docker pull $REG/app:1.4.2          # new host

# Image without registry
docker save myorg/app:1.4.2 | gzip > app.tar.gz
scp app.tar.gz user@newhost:/tmp/ && ssh user@newhost "gunzip -c /tmp/app.tar.gz | docker load"

# Volume data
docker run --rm -v appdata:/data -v $PWD:/backup alpine tar czf /backup/appdata.tgz -C /data .
docker run --rm -v appdata:/data -v $PWD:/backup alpine tar xzf /backup/appdata.tgz -C /data
```

> ***What to say to the interviewer:*** *I treat containers as disposable - the image goes through a registry (or docker save and load if there is no registry), the run configuration comes from a Compose file in git, and volume data is backed up and restored separately since it is not part of the image. Then I start it on the new host, verify, and switch traffic.*

Scenario-Based

Q31. A Docker container keeps restarting (or suddenly stops / crashes) in production. How do you find the root cause?

Also asked as: If a running container suddenly stops, how do you troubleshoot? A container has crashed or exited - how do you figure out what went wrong?

> **In simple words:** Like a car that keeps stalling: check the dashboard warning (exit code), read the trip log (container logs), and check whether it ran out of fuel (memory) or was told to stop by someone else.

Detailed answer

1. **Status and restart count**: `docker ps -a` - look at STATUS and how often it restarts.
2. **Exit code and reason**: `docker inspect` -> `State.ExitCode`, `State.OOMKilled`, `State.Error`.

- `0` = process finished (wrong CMD, app not long-running).
- `1` = app error (config, missing env, crash).
- `125/126/127` = docker run error / not executable / command not found.
- `137` = SIGKILL, usually **OOM killed** or forced kill.
- `139` = segfault. `143` = SIGTERM (normal stop).

3. **Logs**: `docker logs --tail 200 <c>` and `--previous` style checks around the crash time.
4. **Events**: `docker events` or `journalctl -u docker` - who stopped it, OOM, health check failures.
5. **Resources**: `docker stats`, host `dmesg | grep -i oom`, disk full (`df -h`).
6. **Dependencies**: DB/API reachable? DNS? Wrong env or missing secret?
7. **Health check** failing -> restart policy or orchestrator kills it.
8. **Reproduce**: run the same image with `--entrypoint sh` to inspect files, config and permissions.
9. **Fix and prevent**: raise memory limit or fix leak, fix config, add proper health check, alert on restart count.

Example:

```
docker ps -a --filter name=api
docker inspect api --format 'Exit={{.State.ExitCode}} OOM={{.State.OOMKilled}} Err={{.State.Error}}'
# Exit=137 OOM=true Err=
docker logs --since 30m api | tail -50
docker events --since 1h --filter container=api
dmesg -T | grep -i -E "killed process|out of memory"
docker run --rm -it --entrypoint sh myorg/api:1.4.2
```

> ***What to say to the interviewer:*** *I start with docker ps -a and docker inspect to get the exit code and OOMKilled flag, then read docker logs and docker events around the crash. Exit 137 with OOMKilled points to memory, exit 1 to an app or config error, 127 to a bad command; I confirm by running the image with a shell, fix the cause and add alerts on restart count.*

Q32. Your Docker image size has grown from 300MB to 3GB. How would you investigate and optimize it?

> **In simple words:** Your suitcase suddenly weighs ten times more. Open it layer by layer to find what got packed by mistake, then repack properly.

Detailed answer

Investigate first (find what changed):

1. `docker history <image>` - see size per layer; the fat layer shows which instruction caused it.
2. `dive <image>` - browse files added per layer and wasted space.
3. `git diff` the Dockerfile and `.dockerignore` between the 300MB and 3GB versions.

Common culprits:

- `.dockerignore` missing or broken -> `.git`, `node_modules`, datasets, logs copied in.
- Base image changed (alpine -> full ubuntu, or a `-dev` / CUDA image).
- Build tools and caches left in (apt lists, pip/npm cache, compilers).
- Files deleted in a later layer (still stored in the earlier one).
- Large artifacts, dumps or test files copied with `COPY . .`.

Fix:

- Restore/strengthen `.dockerignore`; copy only needed paths.
- Multi-stage build, slim/distroless runtime.
- Clean up in the same RUN; `--no-install-recommends`, `--no-cache-dir`.
- Move big static data to S3 or a volume.
- Add a CI guard that fails the build if the image exceeds a size budget.

Example:

```
docker images myapp
docker history --format "{{.Size}}\t{{.CreatedBy}}" myapp:bad | sort -h | tail -5
# 2.6GB   COPY . .     <- culprit: whole repo incl. data/ and .git
cat .dockerignore
dive myapp:bad

# CI size guard
SIZE=$(docker image inspect myapp:$TAG --format '{{.Size}}')
[ "$SIZE" -gt 524288000 ] && echo "Image > 500MB" && exit 1
```

> ***What to say to the interviewer:*** *I use docker history and dive to find the exact layer that grew, then diff the Dockerfile and .dockerignore against the last good version - usually it is a broken .dockerignore, a heavier base image or leftover build caches. I fix it with multi-stage builds, targeted COPY and same-layer cleanup, and add a size budget check in CI.*

Q33. Your Docker image build time has shot up to 25 minutes. How do you optimize it?

> **In simple words:** If you rewash every dish every time you cook one new dish, dinner takes forever. Reuse what is already clean (cache) and only redo what changed.

Detailed answer

1. **Measure**: build with `--progress=plain` to see which step is slow.
2. **Fix layer order**: copy dependency manifests first, install, then `COPY . .`. If source is copied before the install, every code change reinstalls all dependencies.
3. `.dockerignore`: a huge build context (GBs of `.git`, data) is uploaded every build.
4. **Enable BuildKit** and cache mounts (`--mount=type=cache`) for npm/pip/maven/go caches.
5. **Remote cache in CI**: ephemeral CI agents start empty, so use `--cache-from` / `--cache-to` with a registry cache (ECR) or the CI cache.
6. **Multi-stage parallelism**: independent stages build in parallel with BuildKit.
7. **Pre-built base image** with heavy OS packages, rebuilt weekly instead of every build.
8. **Pin versions** so cache is stable; avoid `apt-get update` busting cache unnecessarily.
9. **Faster runners** / local registry mirror to avoid slow pulls and rate limits.
10. Build only changed services in a monorepo.

Example:

```
# syntax=docker/dockerfile:1
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci
COPY . .
RUN npm run build
```

```
docker buildx build \
  --cache-from type=registry,ref=$REG/app:buildcache \
  --cache-to   type=registry,ref=$REG/app:buildcache,mode=max \
  -t $REG/app:$TAG --push .
```

> ***What to say to the interviewer:*** *I first profile the build with plain progress to find the slow step, then fix layer ordering so dependency installs are cached, trim the build context with .dockerignore, and enable BuildKit cache mounts. In CI I add a registry-backed cache with cache-from and cache-to, which usually brings builds from 25 minutes down to a few minutes.*

Q34. You are building a Docker image but requirements.txt (or package.json) keeps installing an old dependency version. How do you fix it?

> **In simple words:** Docker remembered an old shopping trip and gave you the same old bag instead of going to the shop again. You need to tell it the list changed or force a fresh trip.

Detailed answer

Likely causes:

1. **Layer cache** - the install step was cached. If the file really changed, the `COPY requirements.txt` layer should bust the cache; if it did not, the file used is not the one you edited.
2. **Unpinned versions** (`requests>=2.0`) - with a cached layer you keep the old resolved version; without cache you may get a random new one.
3. **Wrong file copied** - build context path, `.dockerignore`, or a stale file in the repo; `COPY` happens after the install step.
4. **Old base image** already has the package installed (pull a fresh base).
5. **Private mirror / proxy** serving a stale index.
6. CI uses a stale remote cache or an old image tag instead of the new build.

Fix:

- Pin exact versions (`requests==2.32.3`) or use a lock file.
- Copy the dependency file just before the install step.
- Rebuild without cache: `docker build --no-cache` or `--pull` for a fresh base.
- Verify inside the image what version got installed.

Example:

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
docker build --no-cache --pull -t app:fix .
docker run --rm app:fix pip show requests | grep Version
docker run --rm app:fix cat requirements.txt     # confirm the right file got in
```

> ***What to say to the interviewer:*** *It is almost always a cached layer or unpinned versions, so I pin exact versions with a lock file, make sure the dependency file is copied right before the install step, and rebuild with --no-cache --pull. Then I verify the installed version inside the image before pushing.*

Q35. Docker containers are consuming high CPU on the host. How do you identify the root cause?

> **In simple words:** The electricity bill is huge - go room to room with a meter to find which appliance is burning power, then check why it is running non-stop.

Detailed answer

1. **Which container?** `docker stats --no-stream` - CPU %, memory, per container.
2. **Which process inside?** `docker top <c>` or `docker exec <c> top`; on host, `top`/`htop` and map PID to container with `docker inspect`.
3. **Is it real work or a problem?**

- Traffic spike -> legit, scale out.
- Infinite loop, busy retry loop, crash-restart loop (restarts cost CPU).
- Memory near limit -> heavy GC (Java/Node) shows as CPU.
- Crypto-miner in a compromised image (unknown processes!).

4. **Logs** for errors or retry storms: `docker logs --since 10m`.
5. **Host-level**: `iowait`, steal time on a VM, noisy neighbour.
6. **Profile the app** (thread dump, `perf`, language profiler).
7. **Contain**: set `--cpus` limits so one container cannot starve others; restart or scale; fix code.

Example:

```
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
docker top api
docker exec -it api top -b -n 1 | head -15
docker inspect api --format '{{.RestartCount}}'
docker update --cpus 1.5 api          # cap CPU live
```

> ***What to say to the interviewer:*** *I use docker stats to find the container, then docker top or top inside it to find the process, and check logs, restart count and memory pressure to decide if it is real load, a loop, GC thrashing or something malicious. I contain it with CPU limits or scaling, then profile and fix the root cause.*

Q36. Docker shows "no space left on device". How do you resolve it?

Also asked as: If Docker shows "no space available", how do you fix it?

> **In simple words:** The storeroom is full of old boxes nobody uses - old images, stopped containers, leftover build scraps and giant log files. Clear them out and set a rule so it doesn't fill again.

Detailed answer

1. **Confirm what is full**: `df -h` (disk) and `df -i` (inodes). Docker data lives in `/var/lib/docker`.
2. **See Docker usage**: `docker system df -v`.
3. **Clean safely**:

- Stopped containers: `docker container prune`

- Dangling/unused images: `docker image prune -a` (removes images not used by any container)

- Build cache: `docker builder prune`

- Unused volumes: `docker volume prune` - careful, this can delete data.

4. **Huge container logs**: `/var/lib/docker/containers/*/*-json.log` can be GBs; truncate, then set log rotation.
5. **Long-term**:

- Log rotation in `/etc/docker/daemon.json`.

- Scheduled prune job on build agents.

- Move Docker data-root to a bigger disk, or grow the EBS volume.

- Disk usage alerts at 80 percent.

Example:

```
df -h /var/lib/docker
docker system df -v
docker container prune -f
docker image prune -a -f --filter "until=168h"
docker builder prune -f
du -sh /var/lib/docker/containers/*/*-json.log | sort -h | tail -3
truncate -s 0 /var/lib/docker/containers/<id>/<id>-json.log
```

```
# /etc/docker/daemon.json
{ "log-driver": "json-file", "log-opts": { "max-size": "50m", "max-file": "3" } }
```

> ***What to say to the interviewer:*** *I confirm with df and docker system df, then prune stopped containers, unused images and build cache, and truncate oversized container logs - being careful with volume prune since it can delete data. Long term I set log rotation in daemon.json, schedule pruning on build agents and alert on disk usage before it reaches 100 percent.*

Q37. The Docker daemon becomes unavailable on a production server. How would you recover?

> **In simple words:** The engine of the machine stopped. First check whether it is out of fuel (disk), overheated (memory), or has a broken setting, then restart it carefully so the running work isn't lost.

Detailed answer

1. **Confirm**: `docker ps` -> "Cannot connect to the Docker daemon". Check `systemctl status docker` and `containerd`.
2. **Read why**: `journalctl -u docker -n 200 --no-pager`.
3. **Common causes and fixes**:

- Disk full on `/var/lib/docker` -> free space (logs, old images), then start.
- Bad `/etc/docker/daemon.json` (JSON typo after a change) -> validate/fix.
- containerd crashed -> `systemctl restart containerd`.
- Socket permission / stale `docker.pid`.
- Host OOM or kernel/storage-driver error.

4. **Restart**: `systemctl restart docker`. With `"live-restore": true`, running containers keep running while the daemon restarts.
5. **Verify**: containers with `--restart unless-stopped` come back; check app health.
6. **If the host is broken**: drain it from the load balancer and replace it (immutable infra) instead of fixing live.
7. **Prevent**: enable live-restore, monitor the docker service and disk, log rotation, test daemon.json changes.

Example:

```
sudo systemctl status docker containerd
sudo journalctl -u docker --since "30 min ago" --no-pager | tail -40
df -h /var/lib/docker
jq . /etc/docker/daemon.json                  # validate JSON
sudo systemctl restart containerd docker
docker ps
```

```
# /etc/docker/daemon.json
{ "live-restore": true, "log-opts": { "max-size": "50m", "max-file": "3" } }
```

> ***What to say to the interviewer:*** *I check systemctl status and journalctl for docker and containerd to find the cause - usually disk full, a bad daemon.json or containerd failure - fix it and restart the service, then verify containers came back via restart policies. For prevention I enable live-restore so containers survive daemon restarts, monitor the service and disk, and replace unhealthy hosts behind the load balancer.*

Q38. A containerised app runs perfectly on a developer machine but crashes in production. What is your debug checklist?

> **In simple words:** The dish tastes great at home but fails in the restaurant. Same recipe? Same ingredients? Same oven settings? Check every difference between the two kitchens.

Detailed answer

1. **Same image?** Compare image digest, not just tag. `latest` or a locally built image may differ from what CI pushed.
2. **CPU architecture**: built on an ARM Mac (arm64) but prod is amd64 -> `exec format error`. Build with `--platform` / buildx multi-arch.
3. **Config and secrets**: missing env vars, wrong `.env`, secrets not injected in prod.
4. **Network and dependencies**: prod DB/API endpoints, security groups, DNS, TLS certificates, proxy.
5. **Resource limits**: prod memory/CPU limit lower -> OOMKilled (exit 137).
6. **Permissions**: prod runs as non-root or read-only filesystem; app writes to a folder it cannot.

7. **Volumes and files**: dev bind-mounts local files that are not in the image.
8. **Versions**: base image, runtime and dependency versions not pinned.
9. **Logs and exit code**: `docker logs`, `docker inspect` in prod.
10. **Reproduce**: run the exact prod image locally with prod-like env and limits.

Example:

```
docker inspect --format '{{index .RepoDigests 0}}' myapp:1.4.2      # compare dev vs prod
docker image inspect myapp:1.4.2 --format '{{.Architecture}}'      # arm64 vs amd64?
docker logs --tail 100 myapp
docker buildx build --platform linux/amd64,linux/arm64 -t $REG/myapp:1.4.2 --push .
docker run --rm --memory 256m --read-only --user 1000 --env-file prod.env myapp:1.4.2
```

> ***What to say to the interviewer:*** *I first confirm it is the same image digest and architecture, then compare everything environmental - env vars and secrets, network access, resource limits, user permissions and mounted files - using the prod logs and exit code. Then I reproduce by running the exact prod image locally with prod-like limits and config.*

Q39. A container is running but not reachable on its port. How do you debug it?

> **In simple words:** The shop is open inside, but customers can't get in. Check: is there a door to the street (port mapping)? Is the shopkeeper listening at the front counter or only in the back room (0.0.0.0 vs 127.0.0.1)? Is a guard blocking the street (firewall / security group)?

Detailed answer

Work from inside out:

1. **Is the app listening inside the container?** `docker exec <c> netstat -tlnp` (or `ss -tlnp`) and `curl localhost:<port>` from inside.
2. **Bound to the right address?** If the app listens on `127.0.0.1`, it is only reachable from inside the container. It must bind to `0.0.0.0`. This is the most common cause.
3. **Is the port published?** `docker ps` / `docker port <c>` - EXPOSE alone does NOT publish. Needs `-p hostPort:containerPort`, and the container port must match the app's real port.
4. **From the host**: `curl localhost:<hostPort>`, `nc -zv localhost <hostPort>`. Port already used by another process? `ss -tlnp | grep <port>`.
5. **Network mode**: with `--network host` there is no `-p` mapping; with `--network none` nothing works. Container-to-container calls should use the container name on a user-defined network, not `localhost`.
6. **Host firewall / iptables**: `ufw`, `firewalld`, or iptables DOCKER chain rules.
7. **Cloud layer**: Security Group / NACL inbound rule for the host port, load balancer target health.
8. **Logs**: `docker logs` - maybe the app failed to start its listener.

Example:

```
docker ps --format "table {{.Names}}\t{{.Ports}}"
# api   3000/tcp            <- NOT published (no 0.0.0.0:8080->3000/tcp)
docker port api
docker exec api ss -tlnp
# LISTEN 0 511 127.0.0.1:3000   <- bound to loopback only, must be 0.0.0.0
curl -v http://localhost:8080/health
nc -zv <server-ip> 8080
sudo iptables -t nat -L DOCKER -n | grep 8080

docker run -d --name api -p 8080:3000 -e HOST=0.0.0.0 myorg/api:1.4.2
```

What to say to the interviewer: *I go inside out - check the app is listening inside the container and bound to 0.0.0.0 rather than 127.0.0.1, confirm the port is actually published with docker port and maps to the right container port, test with curl and nc from the host, and then check the host firewall and cloud security group. The two most common causes are a loopback bind and a missing or mismatched -p mapping.*